

Project: Student Mental Health & Counseling Support System

NAME	CHAUDHARI DIPAK RAJENDRA
SRN	PES1UG24CS804
SECTION	K

TRIGGERS:-

```
-- Trigger to log appointment status changes
DELIMITER //
CREATE TRIGGER after_appointment_status_change
AFTER UPDATE ON APPOINTMENT
FOR EACH ROW
BEGIN
    IF OLD.status != NEW.status THEN
        INSERT INTO APPOINTMENT_HISTORY (
            appointment_id,
            old_status,
            new_status,
            changed_at
        ) VALUES (
            NEW.appointment_id,
            OLD.status,
            NEW.status,
            NOW()
        );
    END IF;
END//
```

```
mysql> DELIMITER //
mysql> CREATE TRIGGER after_appointment_status_change
-> AFTER UPDATE ON APPOINTMENT
-> FOR EACH ROW
-> BEGIN
->     IF OLD.status != NEW.status THEN
->         INSERT INTO APPOINTMENT_HISTORY (
->             appointment_id,
->             old_status,
->             new_status,
->             changed_at
->         ) VALUES (
->             NEW.appointment_id,
->             OLD.status,
->             NEW.status,
->             NOW()
->         );
->     END IF;
-> END//
ERROR 1359 (HY000): Trigger already exists
```

```
-- Trigger to create student profile after registration
CREATE TRIGGER after_student_registration
AFTER INSERT ON STUDENT
FOR EACH ROW
BEGIN
    INSERT INTO STUDENT_PROFILE (
        student_id,
        last_session_date,
        total_sessions,
        created_at
    ) VALUES (
        NEW.student_id,
        NULL,
        0,
        NOW()
    );
END//
DELIMITER ;
```

```
mysql> CREATE TRIGGER after_student_registration
-> AFTER INSERT ON STUDENT
-> FOR EACH ROW
-> BEGIN
->     INSERT INTO STUDENT_PROFILE (
->         student_id,
->         last_session_date,
->         total_sessions,
->         created_at
->     ) VALUES (
->         NEW.student_id,
->         NULL,
->         0,
->         NOW()
->     );
-> END//
ERROR 1359 (HY000): Trigger already exists
mysql> DELIMITER ;
```

PROCEDURES:-

```
CREATE PROCEDURE BookAppointment(
    IN p_student_id INT,
    IN p_preferred_date DATE,
    IN p_preferred_time TIME,
    IN p_specialization_id INT
)
BEGIN
    DECLARE v_counselor_id INT;
    DECLARE v_appointment_id INT;

    -- Find available counselor with the least appointments on that day
    SELECT c.counselor_id INTO v_counselor_id
```

```

FROM COUNSELOR c
LEFT JOIN APPOINTMENT a ON c.counselor_id = a.counselor_id
    AND DATE(a.appointment_date) = p_preferred_date
WHERE c.specialization_id = p_specialization_id
    AND c.is_available = 1
GROUP BY c.counselor_id
ORDER BY COUNT(a.appointment_id) ASC
LIMIT 1;

-- Create the appointment
IF v_counselor_id IS NOT NULL THEN
    INSERT INTO APPOINTMENT (
        student_id,
        counselor_id,
        appointment_date,
        appointment_time,
        status,
        created_at
    ) VALUES (
        p_student_id,
        v_counselor_id,
        p_preferred_date,
        p_preferred_time,
        'pending',
        NOW()
    );

    SET v_appointment_id = LAST_INSERT_ID();

-- Return the appointment details
SELECT
    a.appointment_id,
    a.appointment_date,
    a.appointment_time,
    c.name as counselor_name,
    a.status
FROM APPOINTMENT a
JOIN COUNSELOR c ON a.counselor_id = c.counselor_id
WHERE a.appointment_id = v_appointment_id;
ELSE
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'No available counselor found for the specified
specialization';
END IF;
END//

```

```

-> GROUP BY c.counselor_id
-> ORDER BY COUNT(a.appointment_id) ASC
-> LIMIT 1;
->
-> -- Create the appointment
-> IF v_counselor_id IS NOT NULL THEN
->   INSERT INTO APPOINTMENT (
->     student_id,
->     counselor_id,
->     appointment_date,
->     appointment_time,
->     status,
->     created_at
->   ) VALUES (
->     p_student_id,
->     v_counselor_id,
->     p_preferred_date,
->     p_preferred_time,
->     'pending',
->     NOW()
->   );
->
-> SET v_appointment_id = LAST_INSERT_ID();
->
-> -- Return the appointment details
-> SELECT
->   a.appointment_id,
->   a.appointment_date,
->   a.appointment_time,
->   c.name as counselor_name,
->   a.status
-> FROM APPOINTMENT a
-> JOIN COUNSELOR c ON a.counselor_id = c.counselor_id
-> WHERE a.appointment_id = v_appointment_id;
-> ELSE
->   SIGNAL SQLSTATE '45000'
->   SET MESSAGE_TEXT = 'No available counselor found for the specified specialization';
-> END IF;
-> END//
Query OK, 0 rows affected (0.11 sec)

```

```

CREATE PROCEDURE GenerateCounselorWorkloadReport(
  IN p_start_date DATE,
  IN p_end_date DATE
)
BEGIN
  SELECT
    c.counselor_id,
    c.name,
    s.name as specialization,
    COUNT(a.appointment_id) as total_appointments,
    SUM(CASE WHEN a.status = 'completed' THEN 1 ELSE 0 END) as
completed_sessions,
    SUM(CASE WHEN a.status = 'cancelled' THEN 1 ELSE 0 END) as
cancelled_sessions,
    ROUND(AVG(CASE WHEN a.status = 'completed' THEN r.rating ELSE NULL
END), 2) as avg_rating
  FROM COUNSELOR c
  LEFT JOIN APPOINTMENT a ON c.counselor_id = a.counselor_id
  AND a.appointment_date BETWEEN p_start_date AND p_end_date
  LEFT JOIN SPECIALIZATION s ON c.specialization_id = s.specialization_id
  LEFT JOIN SESSION_RATING r ON a.appointment_id = r.appointment_id

```

```
GROUP BY c.counselor_id, c.name, s.name
ORDER BY total_appointments DESC;
END//
```

```
mysql> CREATE PROCEDURE GenerateCounselorWorkloadReport(
->     IN p_start_date DATE,
->     IN p_end_date DATE
-> )
-> BEGIN
->     SELECT
->         c.counselor_id,
->         c.name,
->         s.name as specialization,
->         COUNT(a.appointment_id) as total_appointments,
->         SUM(CASE WHEN a.status = 'completed' THEN 1 ELSE 0 END) as completed_sessions,
->         SUM(CASE WHEN a.status = 'cancelled' THEN 1 ELSE 0 END) as cancelled_sessions,
->         ROUND(AVG(CASE WHEN a.status = 'completed' THEN r.rating ELSE NULL END), 2) as avg_rating
->     FROM COUNSELOR c
->     LEFT JOIN APPOINTMENT a ON c.counselor_id = a.counselor_id
->         AND a.appointment_date BETWEEN p_start_date AND p_end_date
->     LEFT JOIN SPECIALIZATION s ON c.specialization_id = s.specialization_id
->     LEFT JOIN SESSION_RATING r ON a.appointment_id = r.appointment_id
->     GROUP BY c.counselor_id, c.name, s.name
->     ORDER BY total_appointments DESC;
-> END//
Query OK, 0 rows affected (0.03 sec)
```

FUNCTIONS:-

```
-- Function to calculate counselor's current workload
CREATE FUNCTION CalculateCounselorWorkload(
    p_counselor_id INT,
    p_start_date DATE,
    p_end_date DATE
)
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
    DECLARE v_total_hours DECIMAL(5,2);

    SELECT COUNT(*) / 8.0 INTO v_total_hours
    FROM APPOINTMENT
    WHERE counselor_id = p_counselor_id
    AND appointment_date BETWEEN p_start_date AND p_end_date
    AND status != 'cancelled';

    RETURN v_total_hours;
END//
```

```
mysql> CREATE FUNCTION CalculateCounselorWorkload(
->     p_counselor_id INT,
->     p_start_date DATE,
->     p_end_date DATE
-> )
-> RETURNS DECIMAL(5,2)
-> DETERMINISTIC
-> BEGIN
->     DECLARE v_total_hours DECIMAL(5,2);
->
->     SELECT COUNT(*) / 8.0 INTO v_total_hours
->     FROM APPOINTMENT
->     WHERE counselor_id = p_counselor_id
->     AND appointment_date BETWEEN p_start_date AND p_end_date
->     AND status != 'cancelled';
->
->     RETURN v_total_hours;
-> END//
Query OK, 0 rows affected (0.01 sec)
```

```
CREATE FUNCTION GetStudentSessionCount(
    p_student_id INT
)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE v_session_count INT;

    SELECT COUNT(*) INTO v_session_count
    FROM APPOINTMENT
    WHERE student_id = p_student_id
    AND status = 'completed';

    RETURN v_session_count;
END//
```

```
mysql> CREATE FUNCTION GetStudentSessionCount(
->     p_student_id INT
-> )
-> RETURNS INT
-> DETERMINISTIC
-> BEGIN
->     DECLARE v_session_count INT;
->
->     SELECT COUNT(*) INTO v_session_count
->     FROM APPOINTMENT
->     WHERE student_id = p_student_id
->     AND status = 'completed';
->
->     RETURN v_session_count;
-> END//
Query OK, 0 rows affected (0.02 sec)
```

```
-- Function to calculate counselor rating
CREATE FUNCTION CalculateCounselorRating(
    p_counselor_id INT
)
RETURNS DECIMAL(3,2)
DETERMINISTIC
BEGIN
    DECLARE v_avg_rating DECIMAL(3,2);

    SELECT AVG(r.rating) INTO v_avg_rating
    FROM APPOINTMENT a
    JOIN SESSION_RATING r ON a.appointment_id = r.appointment_id
    WHERE a.counselor_id = p_counselor_id
    AND a.status = 'completed';

    RETURN COALESCE(v_avg_rating, 0.00);
END//
```

```
mysql> CREATE FUNCTION CalculateCounselorRating(
->     p_counselor_id INT
-> )
-> RETURNS DECIMAL(3,2)
-> DETERMINISTIC
-> BEGIN
->     DECLARE v_avg_rating DECIMAL(3,2);
->
->     SELECT AVG(r.rating) INTO v_avg_rating
->     FROM APPOINTMENT a
->     JOIN SESSION_RATING r ON a.appointment_id = r.appointment_id
->     WHERE a.counselor_id = p_counselor_id
->     AND a.status = 'completed';
->
->     RETURN COALESCE(v_avg_rating, 0.00);
-> END//
Query OK, 0 rows affected (0.02 sec)
```

COMPLEX QUERIES:-

1.

```
-- Query 1: Get comprehensive appointment analytics
SELECT
    d.name AS department,
    s.name AS specialization,
    COUNT(DISTINCT a.student_id) as unique_students,
    COUNT(a.appointment_id) as total_appointments,
    SUM(CASE WHEN a.status = 'completed' THEN 1 ELSE 0 END) as
completed_sessions,
```

```

        SUM(CASE WHEN a.status = 'cancelled' THEN 1 ELSE 0 END) as
cancelled_sessions,
        ROUND(AVG(CASE WHEN a.status = 'completed' THEN r.rating ELSE NULL END),
2) as avg_rating,
        ROUND(COUNT(CASE WHEN a.status = 'cancelled' THEN 1 END) * 100.0 /
COUNT(*), 2) as cancellation_rate
FROM APPOINTMENT a
JOIN STUDENT st ON a.student_id = st.student_id
JOIN COUNSELOR c ON a.counselor_id = c.counselor_id
JOIN DEPARTMENT d ON st.department_id = d.department_id
JOIN SPECIALIZATION s ON c.specialization_id = s.specialization_id
LEFT JOIN SESSION_RATING r ON a.appointment_id = r.appointment_id
GROUP BY d.name, s.name
ORDER BY total_appointments DESC;

```

2.

```

-- Query 2: Student mental health trend analysis
SELECT
    st.student_id,
    st.name as student_name,
    d.name as department,
    COUNT(a.appointment_id) as total_sessions,
    MIN(a.appointment_date) as first_session,
    MAX(a.appointment_date) as last_session,
    ROUND(AVG(mh.stress_level), 2) as avg_stress_level,
    ROUND(AVG(mh.anxiety_level), 2) as avg_anxiety_level,
    STRING_AGG(DISTINCT sp.name, ', ') as specializations_consulted
FROM STUDENT st
JOIN DEPARTMENT d ON st.department_id = d.department_id
JOIN APPOINTMENT a ON st.student_id = a.student_id
JOIN COUNSELOR c ON a.counselor_id = c.counselor_id
JOIN SPECIALIZATION sp ON c.specialization_id = sp.specialization_id
LEFT JOIN MENTAL_HEALTH_ASSESSMENT mh ON a.appointment_id = mh.appointment_id
WHERE a.status = 'completed'
GROUP BY st.student_id, st.name, d.name
HAVING COUNT(a.appointment_id) > 1
ORDER BY avg_stress_level DESC;

```


3.

```
-- Query 3: Counselor performance and availability analysis
WITH CounselorStats AS (
    SELECT
        c.counselor_id,
        c.name,
        s.name as specialization,
        COUNT(a.appointment_id) as total_appointments,
        COUNT(DISTINCT a.student_id) as unique_students,
        ROUND(AVG(CASE WHEN a.status = 'completed' THEN r.rating END), 2) as
avg_rating,
        SUM(CASE WHEN a.status = 'cancelled' THEN 1 ELSE 0 END) as
cancellations,
        c.is_available
    FROM COUNSELOR c
    LEFT JOIN APPOINTMENT a ON c.counselor_id = a.counselor_id
    JOIN SPECIALIZATION s ON c.specialization_id = s.specialization_id
    LEFT JOIN SESSION_RATING r ON a.appointment_id = r.appointment_id
    GROUP BY c.counselor_id, c.name, s.name, c.is_available
)
SELECT
    cs.*,
    ROUND(cancellations * 100.0 / NULLIF(total_appointments, 0), 2) as
cancellation_rate,
    ROUND(unique_students * 100.0 / NULLIF(total_appointments, 0), 2) as
student_retention_rate,
    RANK() OVER (ORDER BY avg_rating DESC) as rating_rank,
    RANK() OVER (ORDER BY total_appointments DESC) as workload_rank
FROM CounselorStats cs
ORDER BY avg_rating DESC;
``` These SQL features provide:
```

#### Above All Sql Features Provides:-

##### 1. Triggers for:

- Logging appointment status changes
- Automatic student profile creation

##### 2. Stored Procedures for:

- Smart appointment booking with counselor assignment
- Generating counselor workload reports

### **3. Functions for:**

- Calculating counselor workload
- Tracking student session counts
- Computing counselor ratings

### **4. Complex Queries for:**

- Department-wise appointment analytics
- Student mental health trend analysis
- Counselor performance metrics